

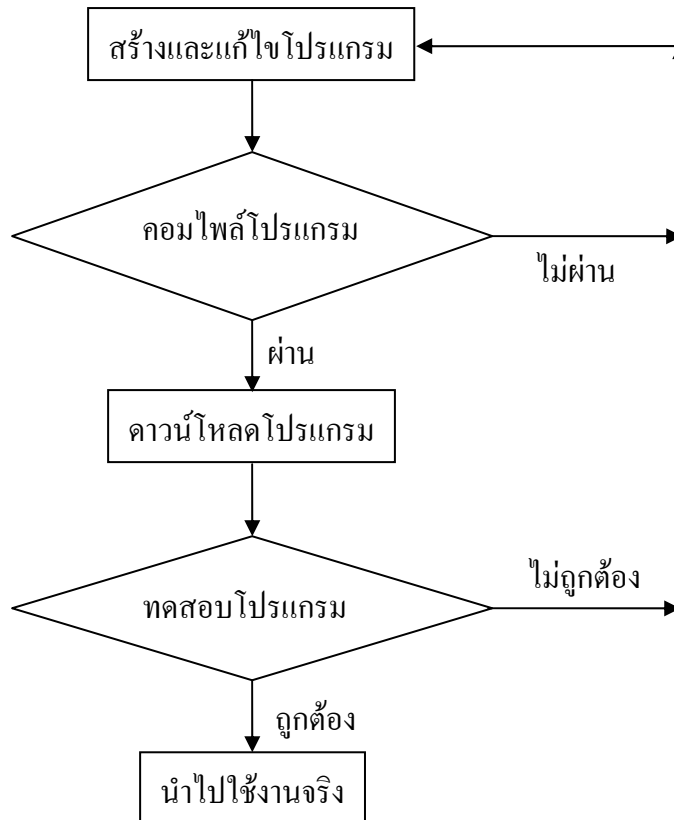
Programming Part

by Werapong Goedsin

ขั้นตอนการพัฒนาโปรแกรมควบคุมหุ่นยนต์

ในการสร้างหุ่นยนต์ให้สามารถทำงานตามความต้องการของเรานั้นเราจำเป็นต้องเขียนโปรแกรมควบคุมฮาร์ดแวร์และตัวรับรู้ (Sensor) ต่างๆ เราอาจคุ้นเคยกับการเขียนโปรแกรมแล้วทำงานบนคอมพิวเตอร์แต่ในการพัฒนาหุ่นยนต์ขนาดเล็กจะใช้คอมพิวเตอร์ควบคุมก็ดูจะไม่เหมาะสม ทางเลือกหนึ่งที่ดีว่าเหมาะสมและยืดหยุ่นในการพัฒนาพอสมควร คือเจ้าคอมพิวเตอร์ขนาดเล็กหรือที่เราเรียกว่า ไมโครคอนโทรลเลอร์ (microcontroller) ซึ่งได้รวบรวมเอา หน่วยประมวลผลกลาง (CPU) หน่วยความจำ (memory) พอร์ตอินพุต/เอาต์พุต (I/O ports) ไม่ว่าจะเป็นพอร์ตชนิดอนุกรมและขนาน และยังสามารถเชื่อมต่อกับอุปกรณ์ภายนอกต่างๆ ได้อีกมากมาย ดังนั้นเราจึงนำไมโครคอนโทรลเลอร์ตัวจิ๋วของเราที่เต็มไปด้วยความสามารถมาควบคุมหุ่นยนต์ ไมโครคอนโทรลเลอร์นั้นยังแบ่งออกเป็นหลายๆ ตระกูลหลายบริษัทให้เราเลือกมาใช้งาน ในที่นี้จะนำเอาไมโครคอนโทรลเลอร์ตระกูล PIC ของบริษัทไมโครชิปมาใช้งานกัน สำหรับการพัฒนาโปรแกรมของไมโครคอนโทรลเลอร์นั้นเราสามารถเลือกใช้ภาษาได้หลายภาษาในการพัฒนา เช่น ภาษาแอสเซมบลี, ภาษาเบสิก, ภาษาโลโก้, ภาษาซี เป็นต้น โดยที่ภาษาที่กล่าวทั้งหมดนั้นยังเป็นภาษาระดับสูงและระดับกลาง ซึ่งไมโครคอนโทรลเลอร์ยังไม่สามารถเข้าใจภาษาเหล่านี้ได้ จำเป็นต้องมีตัวแปลภาษาหรือเรียกว่าคอมไพเลอร์ (compiler) แปลให้เป็นภาษาที่ไมโครคอนโทรลเลอร์สามารถเข้าใจได้ซึ่งเรียกภาษาที่ไมโครคอนโทรลเลอร์เข้าใจว่าภาษาเครื่อง

ภาษาที่เราจะกล่าวถึงในการพัฒนาโปรแกรมจะใช้ภาษาซี โดยใช้คอมไพเลอร์ CCS C ภายใต้ซอฟต์แวร์เครื่องมือพัฒนาภาษาซีรุ่น PCWH ซึ่งเป็นซอฟต์แวร์สำหรับคอมไพล์โปรแกรมภาษาซีที่ใช้กับไมโครคอนโทรลเลอร์ตระกูล PIC ผลิตโดยบริษัท Custom Computer Service ส่วนขั้นตอนในการพัฒนาโปรแกรมควบคุมหุ่นยนต์นั้น สามารถสรุปขั้นตอนคร่าวๆ เป็นแผนภูมิงาน (flow chart) ได้ดังนี้



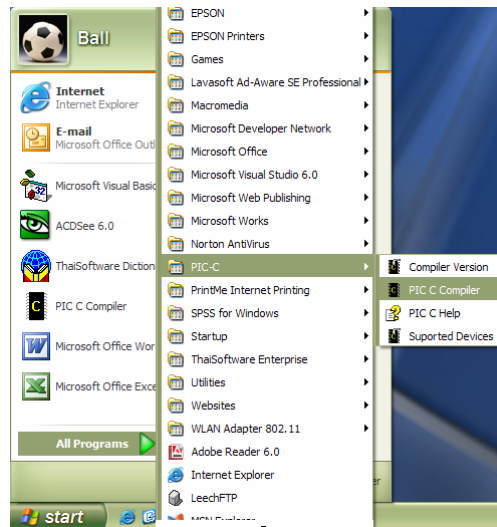
รูปที่ 1 ขั้นตอนการพัฒนาโปรแกรม

การใช้งานโปรแกรม PCWH หรือ PIC C Compiler

PCWH เป็นคอมไพเลอร์ภาษาซีที่ได้รวบรวมเอาเท็กซ์เอดิเตอร์, คอมไพเลอร์, แอสเซมเบลอร์และดีบั๊กเกอร์ทำงานผ่านส่วนพัฒนาร่วม (IDE: Integrated Development Environment) บนระบบปฏิบัติการวินโดวส์ สามารถรองรับไมโครคอนโทรลเลอร์ PIC ครบทุกรุ่นทั้ง 12, 14 และ 16 บิต

ขั้นตอนการใช้งานโปรแกรม

1. เปิด PCWH IDE ขึ้นมาใช้งานโดย ไปที่ start → All Programs → PIC-C → PIC C Compiler ดังรูปที่ 2 หรือคลิกไปที่ไอคอน PIC C Compiler ดังรูปที่ 3 หลังจากโปรแกรม PIC C Compiler ถูกเรียกขึ้นมาใช้งานแล้ว ถ้ามีไฟล์เปิดอยู่ให้ปิดไฟล์ทั้งหมด โดยคลิกไปที่ File → Close all

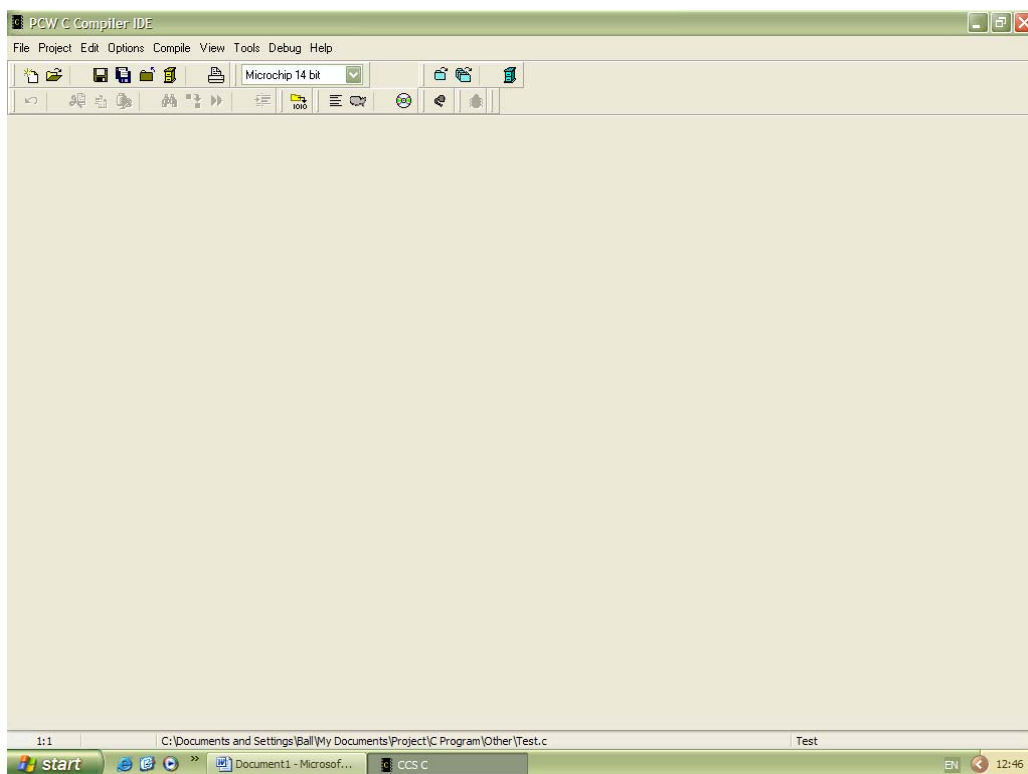


รูปที่ 2 แสดงการเปิดใช้โปรแกรม PIC C Compiler



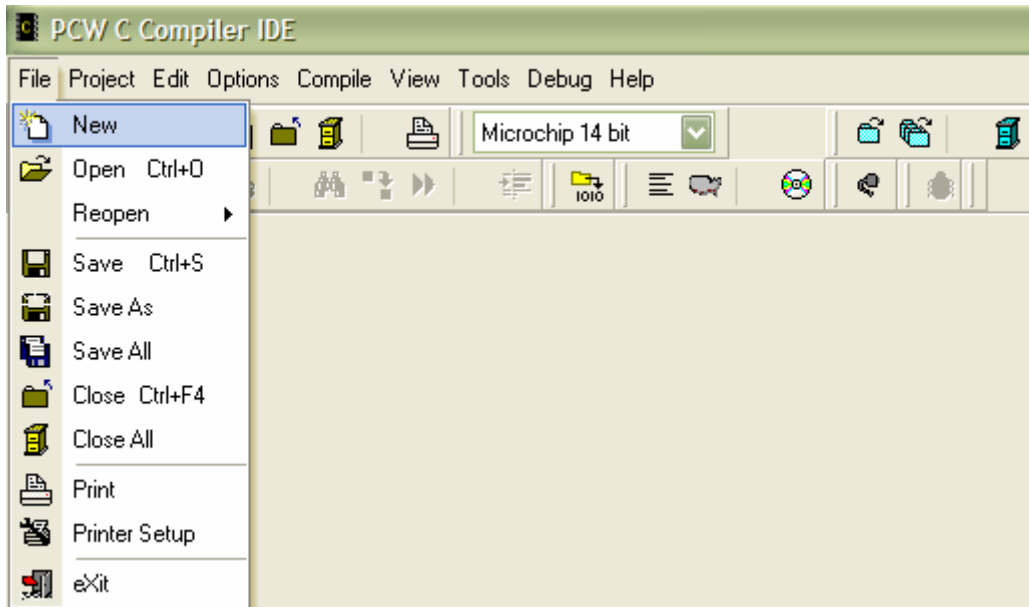
รูปที่ 3 แสดงไอคอนของ PIC C Compiler

2. เมื่อเรียก PIC C Compiler แล้ว จะได้นหน้าต่างของโปรแกรมดังรูปที่ 4

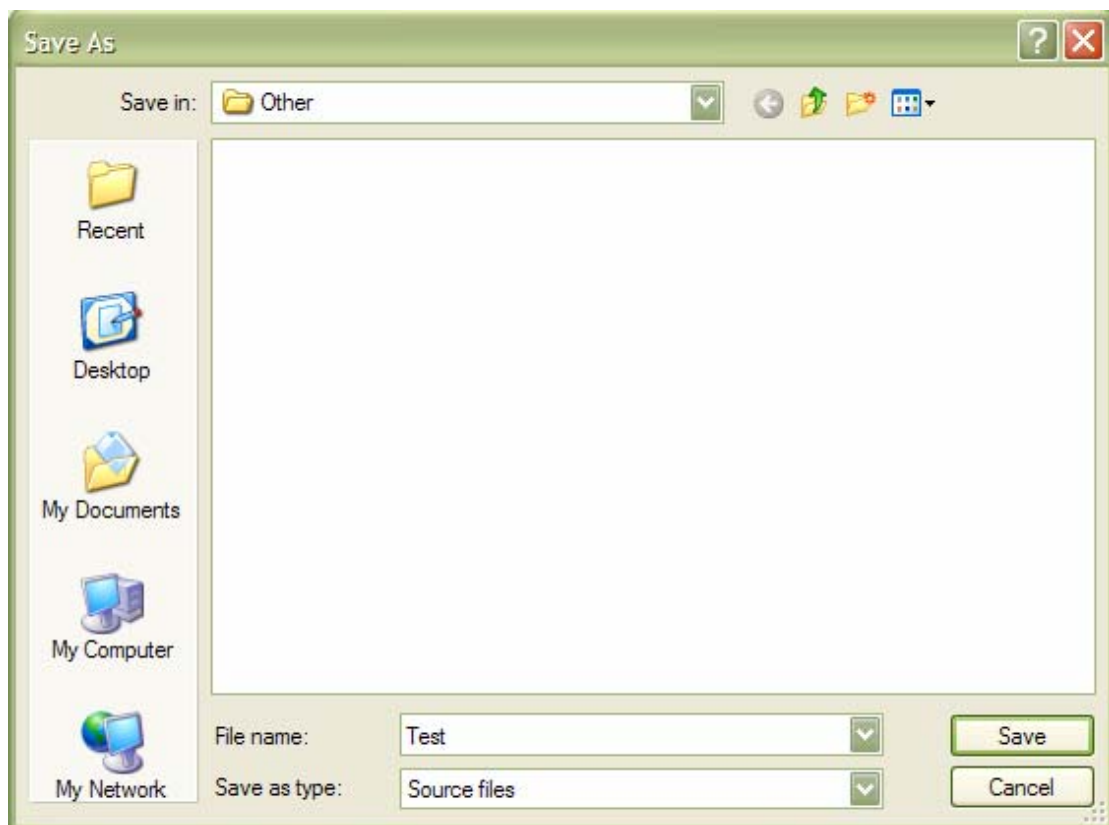


รูปที่ 4 แสดงหน้าต่างของ PCW C Compiler IDE

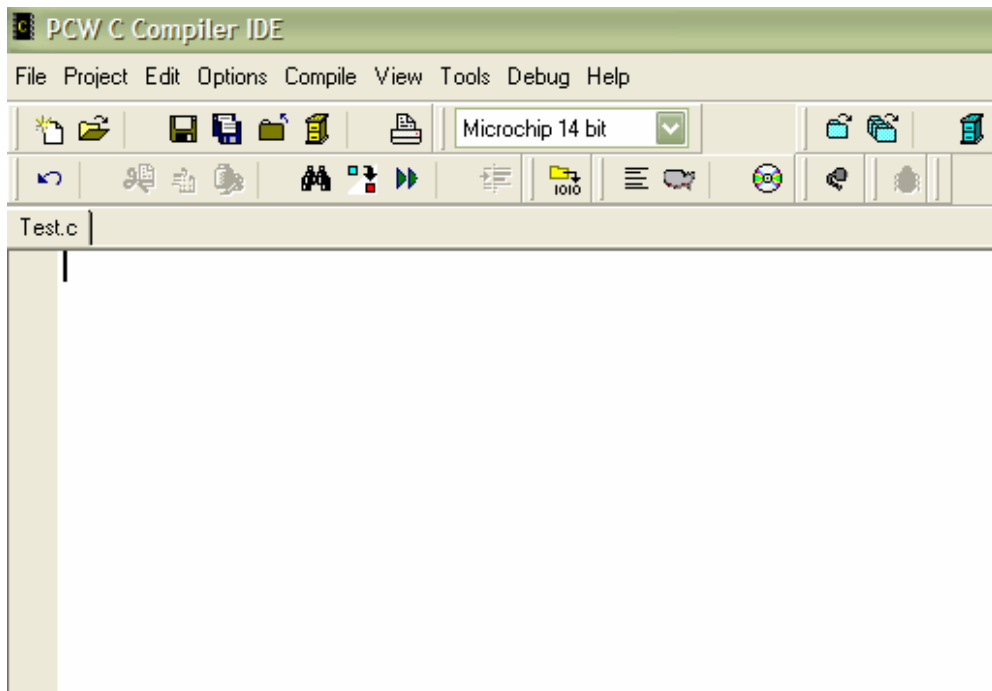
3. สร้างไฟล์ใหม่โดยไปที่เมนู File → New ดังรูปที่ 5 หลังจากนั้นจะมีหน้าต่างใหม่เกิดขึ้นให้ป้อนชื่อไฟล์ (ในที่นี้ตั้งชื่อว่า Test) ในช่อง File Name เก็บไฟล์ไว้ในโฟลเดอร์ที่ต้องการ จากนั้นคลิกปุ่ม Save ดังรูปที่ 6 จากนั้นจะปรากฏหน้าต่างสำหรับเขียนโปรแกรม Test.c ดังรูปที่ 7



รูปที่ 5 แสดงการสร้างไฟล์ใหม่

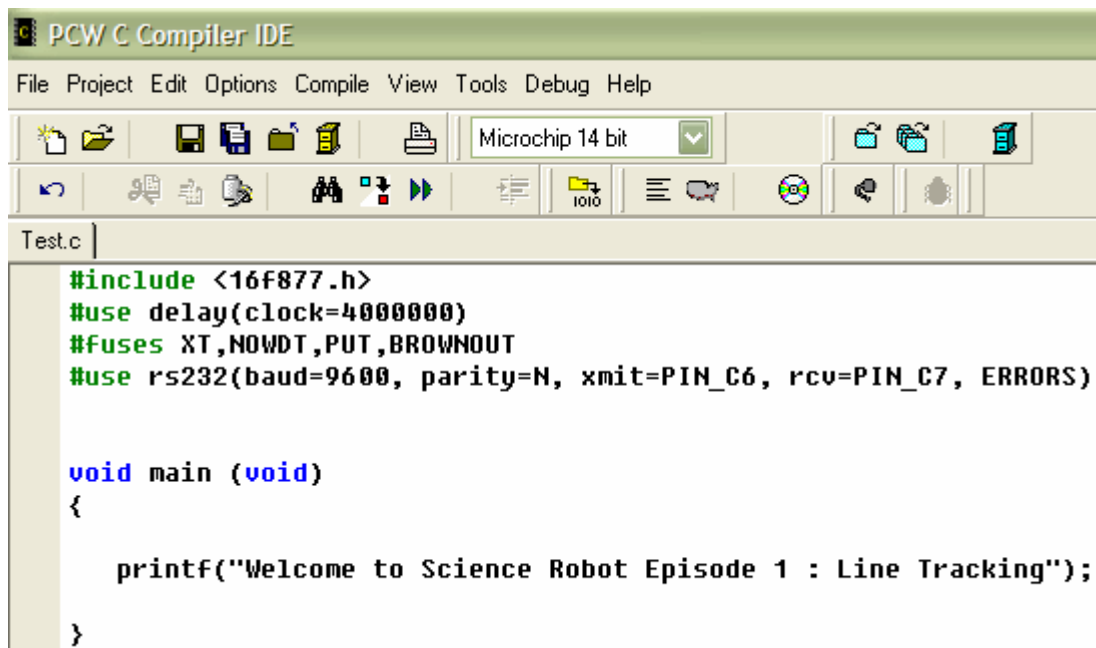


รูปที่ 6 แสดงการตั้งชื่อไฟล์

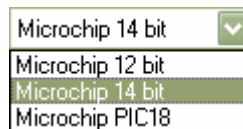


รูปที่ 7 หน้าต่างสำหรับการเขียนโปรแกรม Test.c

4. เขียนโปรแกรมดังอย่างดัง
5. รูปที่ 8 หลังจากเขียนโปรแกรมเสร็จแล้วให้ตรวจสอบหน่วยความจำในการคอมไพล์ในคอมโบบ็อกซ์ (Combo Box) ว่าถ้าไม่ใช่ Microchip 14 bit ให้เลือกไปที่ Microchip 14 bit ดัง
6. รูปที่ 9 (เนื่องจากโปรแกรมตัวอย่างเลือกใช้ PIC16F877 ซึ่งเป็นไมโครคอนโทรลเลอร์ 14 บิต)

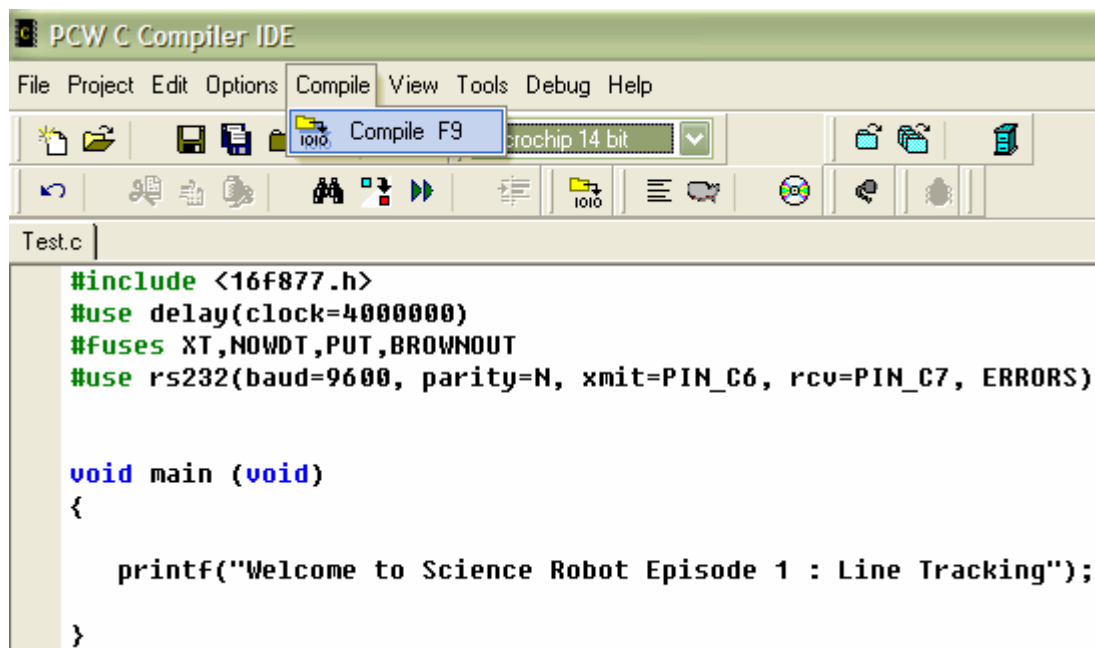


รูปที่ 8 แสดงโปรแกรมตัวอย่าง

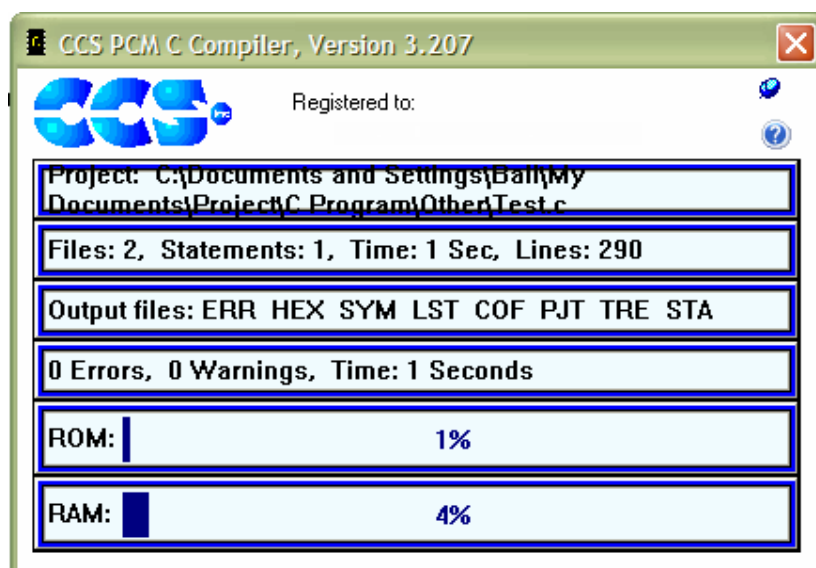


รูปที่ 9 แสดงคอมโบบ็อกซ์เลือกหน่วยความจำในการคอมไพล์โปรแกรม

7. ทำการคอมไพล์โปรแกรมโดยไปที่เมนู Compile → Compile ดังรูปที่ 10 หากสามารถคอมไพล์แล้วไม่เกิดข้อผิดพลาด จะปรากฏหน้าต่างดังรูปที่ 11 ซึ่งเมื่อคอมไพล์เสร็จแล้วจะได้ไฟล์ Test.hex (เนื่องจากเราทำการคอมไพล์ไฟล์ที่ชื่อ Test) เพื่อนำไปดาวน์โหลดลงในไมโครคอนโทรลเลอร์ต่อไป



รูปที่ 10 แสดงการคอมไพล์โปรแกรมที่เขียนแล้ว



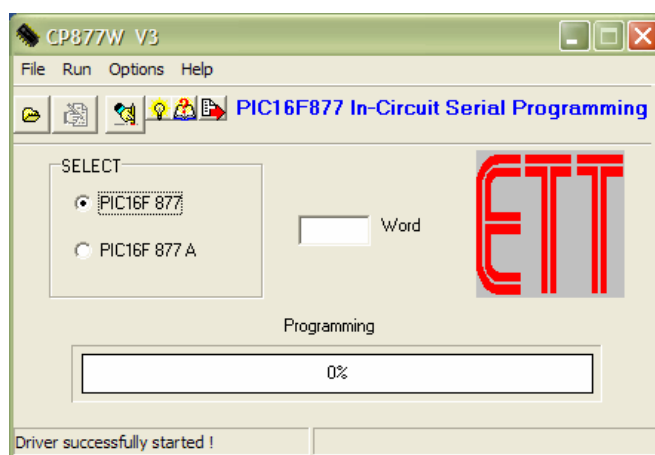
รูปที่ 11 แสดงผลการคอมไพล์

การดาวน์โหลดโปรแกรม

1. ต่อสายคาวนัโหลตระหว่างคอมพิวเตอร์และไมโครคอนโทรลเลอร์ให้เรียบร้อย และตรวจสอบให้ได้ว่าต่อแหล่งจ่ายไฟให้กับไมโครคอนโทรลเลอร์แล้วหรือยัง
2. ดับเบิ้ลคลิกที่ไอคอน CP877WV3 ดัง
3. รูปที่ 12 จะปรากฏหน้าต่างโปรแกรมดังรูปที่ 13

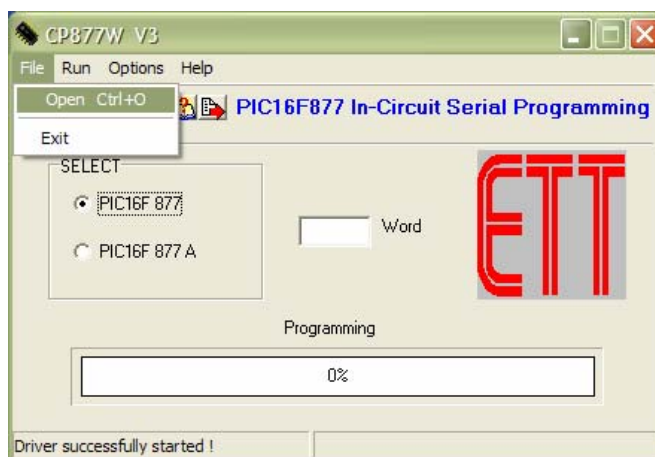


รูปที่ 12 ไอคอน CP877WV3

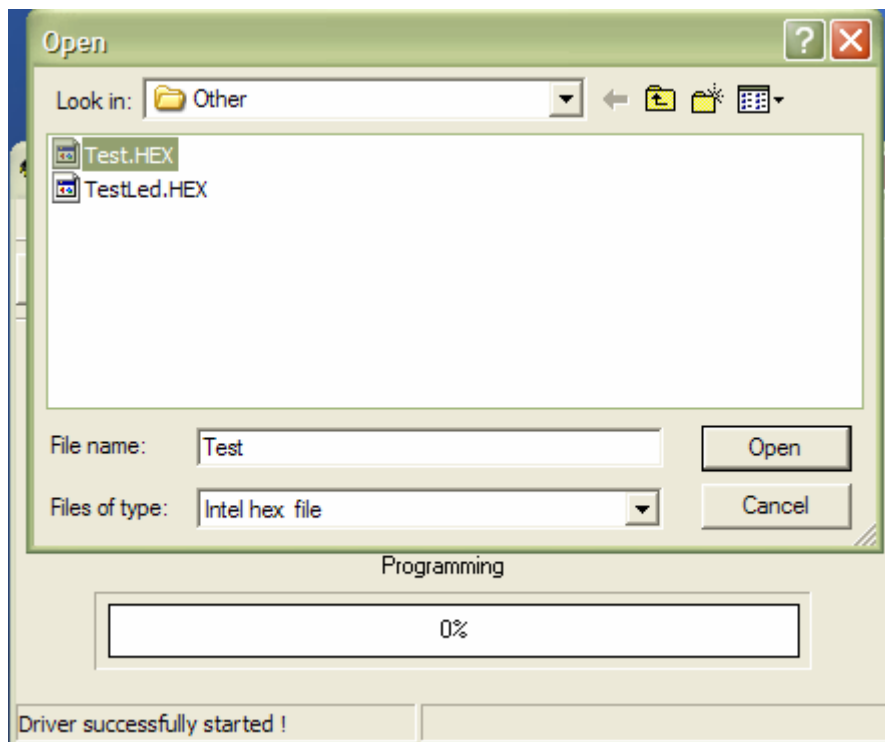


รูปที่ 13 หน้าต่างโปรแกรม CP877W V3

4. ทำการเลือกไฟล์ที่จะดาวน์โหลดลงในไมโครคอนโทรลเลอร์โดยไปที่เมนู File → Open ดังรูปที่ 14 จะปรากฏหน้าต่างขึ้นมา ให้ทำการเลือกไฟล์ เมื่อเลือกเสร็จแล้วให้คลิกที่ Open (ในที่นี้ทำการเลือกไฟล์ Test.hex)

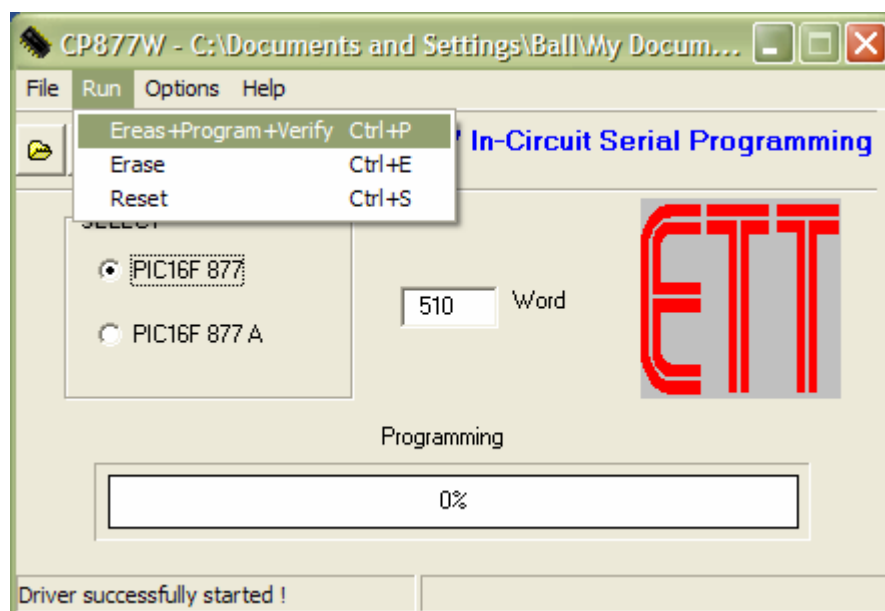


รูปที่ 14 แสดงการเลือกไฟล์



รูปที่ 15 แสดงการเลือกไฟล์

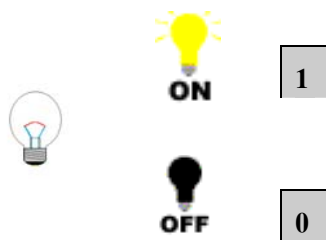
5. ทำการดาวน์โหลดไฟล์โดยไปที่เมนู Run → Erase+Program+Verify ดังรูปที่ 16 เมื่อทำการดาวน์โหลดโปรแกรมเสร็จ จึงนำไมโครคอนโทรลเลอร์ไปใช้งานต่อไป



รูปที่ 16 แสดงการดาวน์โหลดไฟล์

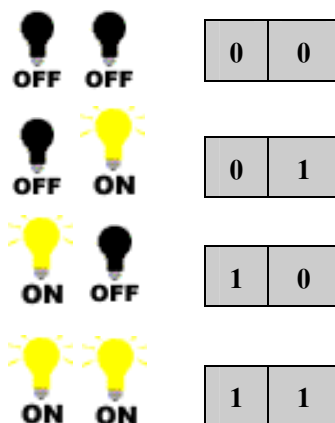
บิตและไบต์

ในคอมพิวเตอร์หรือไมโครคอนโทรลเลอร์ภายในที่ตาเรามองไม่เห็นนั้นองค์ประกอบพื้นฐานของมัน คือ ทรานซิสเตอร์ซึ่งเป็นอุปกรณ์อิเล็กทรอนิกส์ทำหน้าที่เป็นสวิตช์ปิด – เปิด เป็นจำนวนมากมาย โดยที่สวิตช์แต่ละตัวจะเป็นเสมือนหน่วยที่ย่อยที่สุดของระบบคอมพิวเตอร์ เราเรียกหน่วยที่ย่อยที่สุดนี้ว่า “บิต” (bits) เพื่อความเข้าใจที่ง่ายขึ้นให้ลองนึกถึงหลอดไฟ 1 ดวง หลอดไฟที่เราคุ้นเคยกันมีสถานะอยู่สองสถานะ คือ เปิดและปิด หากเราให้สถานะเปิดมีค่าเท่ากับ 1 ปิดมีค่าเท่ากับ 0 หลอดไฟ 1 ดวงจะสามารถสร้างรหัส ได้ 2 แบบ คือ



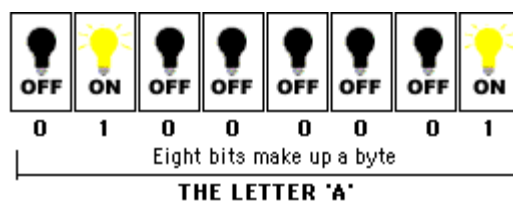
รูปที่ 17 แสดงข้อมูล 1 บิต

หากเรามีหลอดไฟ 2 ดวงจะสามารถแทนรหัสได้เท่ากับ 4 แบบ คือ 2^2 (2×2)



รูปที่ 18 แสดงข้อมูล 2 บิต

ถ้ามีหลอดไฟอยู่จำนวน 8 ดวง ก็จะสามารถสร้างรหัสได้เท่ากับ 256 แบบ คือ 2^8 ($2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2$) เราสามารถนำรหัสเหล่านี้ซึ่งเป็นจำนวนเลขฐานสองนำมาแทนอักษร A – Z, a – z รวมทั้งตัวเลข 0 – 9 เช่น ค่า 01000001 จะแทนตัวอักษร A ดังรูป



รูปที่ 19 แสดงข้อมูล 8 บิตแทนอักษร

ค่า 01000001 เป็นจำนวนเลขฐานสองซึ่งมีค่าเท่ากับ 65 คำนวณได้จาก

$$\begin{aligned}\text{ค่าในฐานสิบ} &= 1*2^0 + 0*2^1 + 0*2^2 + 0*2^3 + 0*2^4 + 0*2^5 + 1*2^6 + 0*2^7 \\ &= 1*1 + 0*2 + 0*4 + 0*8 + 0*16 + 0*32 + 1*64 + 0*128 \\ &= 1+64 \\ &= 65\end{aligned}$$

จะเห็นว่าถ้าเรามีจำนวนทราซิสเตอร์จำนวนมากขึ้นจะทำให้สามารถสร้างรหัสได้มากขึ้น หรือสามารถมีได้หลายบิตนั่นเอง ทำให้มีการคิดหน่วยที่ใหญ่กว่าบิตขึ้น คือ ไบต์ (Byte) โดยที่ 1 ไบต์จะเท่ากับ 8 บิต และนอกจากนี้ยังมีหน่วยอื่นๆ ที่ขนาดใหญ่ขึ้น เช่น กิโลไบต์ มีค่าเท่ากับ 1024 ไบต์, เมกะไบต์ (MB) มีค่าเท่ากับ 1024 กิโลไบต์ เป็นต้น

มาทำความรู้จักภาษาซีกันเถอะ

ในหัวข้อนี้เราจะมาทำความรู้จักกับภาษาซี วิธีการเขียนโปรแกรมให้คอมพิวเตอร์สามารถเข้าใจในสิ่งที่เราเขียนขึ้นมาได้ แต่ก่อนที่เราจะเริ่มต้นเขียนโปรแกรมภาษาใดก็ตามสิ่งที่เราต้องทำความเข้าใจเป็นสิ่งแรกคือ ลักษณะโครงสร้างของภาษานั้นกันก่อน โครงสร้างของภาษาซีมาตรฐาน (ANSI Standard C) จะประกอบไปด้วยรายละเอียดดังนี้

พรีโปรเซสเซอร์ไดเรกทีฟ (Preprocessor Directives)

เป็นชุดคำสั่งที่ใช้ในการจัดเตรียมข้อมูลในการประมวลผล ซึ่งจะเกิดขึ้นก่อนมีการคอมไพล์พรีโปรเซสเซอร์ไดเรกทีฟจะต้องเริ่มต้นด้วยเครื่องหมาย # แล้วตามด้วยคำสั่งที่ต้องการใช้งาน คำสั่งที่ใช้บ่อย คือ **#define** และ **#include** เช่น

```
#include<16F877.h>
```

```
#define BIT 8
```

คำสั่ง `#include<16F877.h>` เป็นคำสั่งในการรวมเอาไฟล์ 16F877.h เข้ามาคอมไพล์ร่วมกับโปรแกรมที่เราเขียนขึ้น ส่วนคำสั่ง `#define BIT 8` เป็นการประกาศค่าคงที่ ให้ BITS มีค่าเท่ากับ 8

ตัวแปร (Variable)

ตัวแปร คือ ชื่อที่ผู้พัฒนาโปรแกรมสร้างหรือประกาศขึ้นมาเพื่อใช้เก็บข้อมูลชั่วคราว โดยข้อมูลที่เก็บในตัวแปรสามารถเปลี่ยนแปลงได้ขณะที่เราทำการรันโปรแกรม การประกาศใช้งานตัวแปรนั้นจะเป็นการจองหน่วยความจำ ขนาดของหน่วยความจำที่ทำการจองนั้นจะขึ้นอยู่กับชนิดข้อมูลของตัวแปร โดยในการใช้งานตัวแปรทุกครั้งจะต้องมีการกำหนดชนิดของตัวแปรซึ่งจะอธิบายในหัวข้อการใช้งานตัวแปร

ส่วนประกาศ (Declaration)

เป็นส่วนที่ใช้ประกาศชื่อและคุณสมบัติชนิดของตัวแปร หรือฟังก์ชันที่ใช้ในโปรแกรม ก่อนที่จะมีการใช้งานตัวแปรและฟังก์ชัน จะต้องทำการประกาศก่อนเสมอ การเข้าถึงเพื่อจะนำตัวแปรมาใช้มีสองรูปแบบ คือ ตัวแปรแบบโกลบอล ตัวแปรชนิดนี้หลังจากประกาศตัวแปรจะสามารถใช้งานได้ในทุกส่วนของโปรแกรม ส่วนอีกแบบคือ ตัวแปรแบบโลคอล จะมีการประกาศใช้เฉพาะในฟังก์ชัน นอกฟังก์ชันที่เราประกาศจะไม่สามารถใช้ตัวแปรชนิดนี้ได้

การกำหนดค่า (Definition)

เป็นการกำหนดค่าตัวแปรหรือรายละเอียดของฟังก์ชัน นอกจากนี้การกำหนดค่าจะเป็นการจองพื้นที่หน่วยความจำในการเก็บค่าต่างๆ ของตัวแปร

นิพจน์ (Expression)

นิพจน์คือ การกระทำระหว่างตัวดำเนินการ (Operators) กับตัวถูกดำเนินการ (Operands) เพื่อให้เกิดค่าใหม่ออกมาค่าหนึ่ง

คำสั่งการทำงาน (Statement)

คำสั่งการทำงานจะเป็นไปตามรูปแบบที่คอมไพเลอร์ได้กำหนดจัดเตรียมไว้เพื่อสั่งงานต่างๆ หรือควบคุมการไหลของโปรแกรม

ฟังก์ชัน (Function)

ฟังก์ชันเป็นส่วนประกอบของโปรแกรมที่กำหนดให้ทำงานอย่างใดอย่างหนึ่งจนเสร็จสิ้น โดยภายในฟังก์ชันจะประกอบไปด้วย การประกาศการใช้งานตัวแปร, การกำหนดค่าให้ตัวแปร, นิพจน์และคำสั่งการทำงาน

ฟังก์ชันหลัก (main function)

ในการเขียนโปรแกรมภาษาซีนั้นจะต้องมีฟังก์ชันหลักทุกครั้งเพราะการทำงานของโปรแกรมนั้นจะมีการเริ่มต้นที่ฟังก์ชันนี้ ภายในฟังก์ชันหลักจะมีการเรียกใช้งานฟังก์ชันอื่นๆ ไม่ว่าจะเป็นฟังก์ชันที่ผู้ใช้สร้างขึ้นเองหรือจะเป็นฟังก์ชันที่คอมไพเลอร์ได้จัดเตรียมไว้ให้ ทุกครั้งที่เรียกใช้งานคำสั่งหรือฟังก์ชันต่างๆ จะตามด้วยเครื่องหมาย ; เสมอเพื่อเป็นการบอกว่าจบคำสั่งแล้ว

ตัวอย่างโปรแกรมแสดงโครงสร้างของภาษาซี

```
/*-----  
                                ฟรีโปรเซสเซอร์ไคเร็กทีฟ  
----- */  
  
#include<16F877.h>  
#use delay (clock = 4000000)  
#fuses XT, NOWDT  
/*-----  
                                ส่วนประกาศแบบโกลบอล  
----- */  
  
int loop = 20;  
unsigned int STEP_1[4] = {0x80, 0x40, 0x20, 0x10};  
unsigned int STEP_2[4] = {0x10, 0x20, 0x40, 0x80};  
/*-----  
                                ฟังก์ชันต้นแบบ (Prototype function)  
----- */  
  
void turn_right (int steps);  
void turn_left (int steps);
```

```

/*-----
                                     ฟังก์ชันหลัก
----- */

main ()
{
    int i;
    for (i = 1; i <= loop; i++)
    {
        turn_right(5);
        delay_ms(200);
        turn_left(10);
    }
} // จบฟังก์ชันหลัก

/*-----
                                     รายละเอียดฟังก์ชัน
----- */

void turn_right (int steps)
{
    int i, j;
    for (i = 1; i < steps; i++)
    {
        for (j = 0; j < 4; j++)
        {
            output_c( STEP_2[j]);
            delay_ms(50);
        }
    }
}

```

```

void turn_left (int steps)
{
    int i, j;
    for (i = 1; i < steps; i++)
    {
        for (j = 0; j < 4; j++)
        {
            output_c( STEP_2[j]);
            delay_ms(50);
        }
    }
}

```

คอมเมนต์ (Comment)

คอมเมนต์ คือ ข้อความหมายเหตุ โดยที่คอมไพเลอร์จะไม่นำข้อความส่วนนี้มาคอมไพล์ เรานิยมใช้หมายเหตุในการอธิบายการทำงาน, ความหมาย, วัตถุประสงค์ของคำสั่งหรือฟังก์ชัน เพื่อให้ผู้อื่นหรือแม้แต่ผู้พัฒนาสามารถเข้าใจ สามารถแก้ไขปรับปรุงโปรแกรมภายหลังได้สะดวกขึ้น รูปแบบการใช้งานคอมเมนต์มี 2 แบบ คือ

- แบบบรรทัดเดียว

จะใช้เครื่องหมาย // เช่น

// begin of main function

หลังเครื่องหมาย // จะเป็นคอมเมนต์คอมไพเลอร์จะไม่นำข้อความส่วนนี้มาทำการคอมไพล์

- แบบหลายบรรทัด

จะใช้เครื่องหมาย /* ตามด้วยคอมเมนต์ที่พิมพ์ได้มากกว่า 1 บรรทัดเมื่อจบข้อความจะต้องตามด้วยเครื่องหมาย */ เช่น

```

/* CCS C Compiler for PIC Microcontroller
and PIC microcontroller in Robot */

```

ค่าคงที่ (Constants)

ค่าคงที่ คือ ค่าที่ไม่สามารถเปลี่ยนแปลงได้ในขณะทำการรันโปรแกรม ประโยชน์ของค่าคงที่ เช่น ค่าที่เราต้องการใช้หลายๆ ที่ในโปรแกรมเพื่อสะดวกในการแก้ไขค่าในภายหลังเราจะประกาศค่านั้นเป็นค่าคงที่ เมื่อต้องการเปลี่ยนค่า เราสามารถเปลี่ยนค่าเหล่านี้เพียงทีเดียวที่ส่วนประกาศค่าคงที่เท่านั้น การประกาศค่าคงที่ในภาษาซีจะใช้ไคเร็กที่ฟ **#define** ในการกำหนดค่าคงที่ เช่น

```
#define PI 3.142
```

เราสามารถค่า PI ไปใช้ในโปรแกรมได้โดยที่ค่า PI จะมีค่าเท่ากับ 3.142 นอกจากนี้ในภาษาซียังมีการใช้ค่าคงที่อีกแบบ โดยการใช้คำสงวน (Key Words) Const เช่น

```
float const PI = 3.142;
```

การใช้งานตัวแปร

ก่อนที่จะทำการเขียนโปรแกรมภาษาซี เราจะต้องมาทำความเข้าใจกับการใช้งานตัวแปร ชนิดข้อมูลที่ใช้ในการสร้างตัวแปร

การประกาศตัวแปร

การประกาศตัวแปรมีรูปแบบการประกาศดังนี้

ชนิดของข้อมูล ชื่อตัวแปร [,.....];

เช่น

```
float a;          //ประกาศตัวแปรประเภทจำนวนจริง (ทศนิยม)
```

ถ้าต้องการประกาศตัวแปรชนิดเดียวกันมากกว่า 1 ตัว ต้องใช้เครื่องหมายคอมม่า (,) คั่นแยกตัวแปร เช่น

```
int x,y,z;        //ประกาศตัวแปร x, y, z เป็นตัวแปรแบบจำนวนเต็ม
```


ในการใช้งานตัวแปรให้ถูกชนิดของข้อมูลมีความจำเป็นอย่างยิ่งในความถูกต้องและรวดเร็วในการประมวลผล ดังนั้นเราจะมาทำความรู้จักกับชนิดของข้อมูลกัน

ชนิดข้อมูล	ขนาด	ค่าอยู่ในช่วง
Unsigned	8 บิต แบบ ไม่คิดเครื่องหมาย	0 ถึง 255
unsigned int		
Int		
Char		
int8		
Long	16 บิต แบบ ไม่คิดเครื่องหมาย	0 ถึง 65535
long int		
int16		
signed	8 บิต แบบ คิดเครื่องหมาย	-128 ถึง 127
signed int		
signed int8		
signed long	16 บิต แบบ คิดเครื่องหมาย	-32768 ถึง 32767
signed int16		
int32	32 บิต แบบ ไม่คิดเครื่องหมาย	0 ถึง 4294967295
signed int32	32 บิต แบบ ไม่คิดเครื่องหมาย	-2×10^9 ถึง 2×10^9
float	32 บิต แบบ ทศนิยม	3.4×10^{-38} ถึง 3.4^{38}
short	1 บิต	0 ถึง 1
short int		
int1		

กฎการตั้งชื่อตัวแปร

ในการสร้างตัวแปรขึ้นมาใช้งานให้ถูกต้องนั้นจะต้องทำตามเงื่อนไขดังต่อไปนี้

- จะต้องขึ้นต้นด้วยอักขระ a ถึง z , A ถึง Z หรือ ชิดล่าง (under score) เท่านั้น ไม่สามารถขึ้นต้นด้วยตัวเลข
- ห้ามเว้นวรรค
- ห้ามตั้งชื่อตัวแปรซ้ำกันภายในขอบเขตการเข้าถึงเดียวกัน
- ชื่อตัวแปรไม่ควรยาวเกินไป และควรสื่อความหมายในการใช้งานตัวแปร

- ห้ามใช้ชื่อตัวแปรซ้ำกับคำสงวนของภาษาซี

คำสงวน (Key Words)

คำสงวนในภาษาซี ประกอบด้วย

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

การจัดการกับตัวเลข

ตัวดำเนินการที่เกี่ยวข้องกับจำนวนในภาษาซีมีอยู่หลายตัวด้วยกัน เพื่อความเข้าใจง่ายขึ้น ขอสรุปเป็นตาราง โดยกำหนดตัวแปร $a = 3$, $b = 17$

ตัวดำเนินการ	ชื่อตัวดำเนินการ	ตัวอย่าง	ผลลัพธ์
+	บวก	$a + b$	20
-	ลบ	$b - a$	14
*	คูณ	$a * b$	51
/	หาร (ไม่เอาเศษ)	b / a	5
%	หารเอาเฉพาะเศษ	$b \% a$	2
-	นิเสธ	$-a$	-3
--	ลดค่า	$--a$	2
++	เพิ่มค่า	$++a$	4

ตัวดำเนินการเปรียบเทียบและลอจิก

ผลลัพธ์ที่ได้จากตัวดำเนินการประเภทนี้จะได้ผลลัพธ์ออกมาเป็นจริง (true) หรือเท็จ (false)

ตัวดำเนินการ	ชื่อตัวดำเนินการ	ตัวอย่าง	ผลลัพธ์
<	น้อยกว่า	$a < b$	จริง (true) หรือมีค่าเท่ากับ 1
<=	น้อยกว่าหรือเท่ากับ	$a <= b$	จริง (true) หรือมีค่าเท่ากับ 1
>	มากกว่า	$a > b$	เท็จ (false) หรือมีค่าเท่ากับ 0
>=	มากกว่าหรือเท่ากับ	$a >= b$	เท็จ (false) หรือมีค่าเท่ากับ 0
==	เท่ากับ	$a == b$	เท็จ (false) หรือมีค่าเท่ากับ 0
!=	ไม่เท่ากับ	$a != b$	จริง (true) หรือมีค่าเท่ากับ 1
&&	และ (Logical AND)	$(a > b) \&\& (b < a)$	เท็จ (false) หรือมีค่าเท่ากับ 0
	หรือ (Logical OR)	$(a > b) (4 > a)$	จริง (true) หรือมีค่าเท่ากับ 1
!	ไม่ใช่ (not)	$!(20 < a)$	จริง (true) หรือมีค่าเท่ากับ 1

ตัวดำเนินการระดับบิต

ตัวดำเนินการ	ชื่อตัวดำเนินการ	ตัวอย่าง	ผลลัพธ์
&	แอนด์ (Bitwise AND)	b&a	1
	ออร์ (Bitwise OR)	b a	19
^	เอ็กคลูซีฟ – ออร์ (Bitwise XOR)	b^a	18
<<	เลื่อนบิตไปทางซ้าย (Shift Left)	b<<a	136
>>	เลื่อนบิตไปทางขวา (Shift Right)	b>>a	2
~	กลับค่าบิต (Logical Complement)	~a	252

การเลือกทำ

เป็นคำสั่งที่เลือกทำอย่างใดอย่างหนึ่งในการทำงานซึ่งจะขึ้นอยู่กับเงื่อนไขที่ทำการตรวจสอบ

การเลือกทำแบบทางเดียว หรือ คำสั่ง IF

เป็นการตรวจสอบเงื่อนไข ถ้านิพจน์เป็นจริงก็จะเข้าไปทำชุดคำสั่งคำสั่งตามต้องการ

รูปแบบการใช้งาน

```
if (เงื่อนไข)
{
    ชุดคำสั่งคำสั่ง;
}
```

ตัวอย่าง

```
if ((a%2) == 0)
```

```
{
    output_high(PIN_C0);
}
```

เป็นการตรวจสอบว่าตัวแปร a เป็นจำนวนคู่ (หารด้วยสองลงตัว) ถ้าเป็นคู่จะส่งลอจิกสูงไปยังพอร์ต C (Pin C0) ในที่นี้หาก Pin C0 ต่ออยู่กับ ไดโอดเปล่งแสง (LED) ก็จะทำให้ไดโอดเปล่งแสง สว่าง

การเลือกทำสองทิศทาง (IF – ELSE)

จะเป็นการตรวจสอบเพื่อทำงานอย่างใดอย่างหนึ่งถ้าเงื่อนไขเป็นจริงก็จะทำงานอย่างหนึ่ง (ชุดคำสั่งที่ 1) แต่ถ้าหากเงื่อนไขเป็นเท็จก็จะไปทำงานอีกอย่างหนึ่ง (ชุดคำสั่งที่ 2)

รูปแบบการใช้งาน

```
if (เงื่อนไข)
{
    ชุดคำสั่งที่ 1;
}
else
{
    ชุดคำสั่งที่ 2;
}
```

ตัวอย่าง

```
if ((a%2) == 0)
{
    output_high(PIN_C0);
}
else
{
    output_high(PIN_C1);
}
```

เป็นการตรวจสอบเงื่อนไขเพิ่มเติมว่าถ้า a ไม่ได้เป็นจำนวนคู่ก็จะส่งลอจิกสูงไปยังพอร์ต C (Pin C1) ในที่นี้หาก Pin C1 ต่ออยู่กับ ไดโอดเปล่งแสง (LED) ก็จะทำให้ไดโอดเปล่งแสง สว่าง (ไดโอดเปล่งแสงคนละดวงกับถ้าเงื่อนไขเป็นจริง)

การเลือกทำแบบหลายทางเลือก

1. if – else if

เราสามารถใช้ if – else if เพื่อเป็นการเลือกทำแบบหลายทางเลือก
ตัวอย่าง

```
if (x >= 90)
{
    printf("EXCELLENCE\n\r");
}
else if (x >=70)
{
    printf("GOOD\n\r");
}
else if (x >=50)
{
    printf("FAIR\n\r");
}
else
{
    printf("FAIL\n\r");
}
```

จากตัวอย่างเป็นการตรวจสอบค่า x ว่ามีค่ามากกว่าหรือเท่ากับ 90 หรือไม่ ถ้าใช่จะแสดงข้อความ **EXCELLENCE** ทางจอภาพ ถ้าไม่ใช่ก็จะทำการตรวจสอบว่าค่า x มีค่ามากกว่าหรือเท่ากับ 70 หรือไม่ ถ้าใช่ก็จะแสดงข้อความ **GOOD** ทางจอภาพ แต่ถ้าไม่ใช่จะทำการตรวจสอบอีกครั้งว่าค่า x มีค่ามากกว่า 50 หรือไม่ถ้าใช่ก็จะแสดงข้อความ **FAIR** แต่ถ้าไม่ใช่ก็จะแสดงข้อความ **FAIL** ทางจอภาพ

2. Switch - Case

รูปแบบการใช้งาน

```
switch (expr)
{
    case ค่าคงที่ 1 : ชุดคำสั่ง 1;
                    break;
    case ค่าคงที่ 2 : ชุดคำสั่ง 2;
                    break;
    case ค่าคงที่ 3 : ชุดคำสั่ง 3;
                    break;
    .....
    default: ชุดคำสั่ง;
}
```

ตัวอย่าง

```
switch (move)
{
    case 1 : backward();
            break;
    case 2 : turn_right();
}
```

```

        break;

    case 3 :turn_left();

        break;

    default: forward();

}

```

จากตัวอย่างเป็นการตรวจสอบเงื่อนไขจากค่าตัวแปร move โดยที่ถ้าค่าของ move มีค่าเท่ากับ 1 ก็จะมีการเรียกใช้ฟังก์ชัน backward() ซึ่งจะทำให้หุ่นยนต์ถอยหลัง จากนั้นจะทำคำสั่ง break เพื่อออกจากคำสั่งการเลือกทำ Switch - Case ในกรณีที่ค่า move มีค่าเท่ากับ 2 จะมีการเรียกใช้ฟังก์ชัน turn_right() ซึ่งเป็นคำสั่งควบคุมให้หุ่นยนต์เลี้ยวขวา จากนั้นจะทำคำสั่ง break เพื่อออกจากคำสั่งการเลือกทำ Switch - Case กรณีถัดไปคือ ถ้า move มีค่าเท่ากับ 3 จะมีการเรียกใช้ฟังก์ชัน turn_left() เพื่อสั่งให้หุ่นยนต์เลี้ยวซ้าย จากนั้นจะทำคำสั่ง break เพื่อออกจากคำสั่งการเลือกทำ Switch - Case หากไม่ตรงกับเงื่อนไขใดที่กล่าวมาก็จะเรียกว่าเป็นค่าปกติหรือค่า default ก็จะสั่งให้หุ่นยนต์เดินหน้าโดยใช้ฟังก์ชัน forward() แล้วออกจากคำสั่งการเลือกทำ Switch - Case โดยไม่ต้องมีคำสั่ง break เพราะเป็นกรณีสุดท้ายไม่จำเป็นต้องใช้คำสั่งในการกระโดดเพื่อออกจากคำสั่งการเลือกทำ

คำสั่งการทำซ้ำ

คำสั่งการทำซ้ำจะเป็นคำสั่งที่ทำให้โปรแกรมทำงานอย่างใดอย่างหนึ่งซ้ำๆ อยู่ในรูปจนกว่าจะตรงตามเงื่อนไขที่ผู้เขียนโปรแกรมกำหนด การทำซ้ำในภาษาซีมีอยู่หลายคำสั่งแต่ละคำสั่งอาจมีวัตถุประสงค์และวิธีใช้งานที่แตกต่างกัน แต่ละคำสั่งมีรายละเอียดดังนี้

คำสั่ง For

มักจะใช้กับการทำซ้ำที่มีจำนวนครั้งที่แน่นอนและจะหยุดทำซ้ำเมื่อเงื่อนไขเป็นเท็จ

รูปแบบการใช้งาน

```

for (exp1; exp2; exp3)
{
    ชุดคำสั่ง;
}

```

โดยที่

exp1 เป็นนิพจน์ที่กำหนดค่าเริ่มต้นให้กับตัวแปรที่ใช้ในการทำซ้ำ

exp2 เป็นนิพจน์ที่ใช้ในการตรวจสอบเงื่อนไขเพื่อหยุดการทำซ้ำ

exp3 เป็นนิพจน์ที่ใช้เพิ่มหรือลดค่าให้กับตัวแปรควบคุมจำนวนการทำซ้ำ

ตัวอย่าง

```
int i;           //ประกาศตัวแปร i เป็นจำนวนเต็ม
for(i = 1; i <= 5; i++)
{
    printf("Number %d\n",i);
}
printf("End Program");
```

เป็นการวนรอบโดยเริ่มต้นให้ตัวแปร i มีค่าเท่ากับ 1 โดยแต่ละรอบของการทำซ้ำ จะมีการเพิ่มค่า i อีก 1 ด้วยคำสั่ง i++ และจะทำซ้ำต่อเมื่อ i ยังน้อยกว่าหรือเท่ากับ 5 ซึ่งก็คือ จะหยุดทำซ้ำเมื่อ i เพิ่มค่าจนเท่ากับ 5 นั่นเอง

ผลลัพธ์

จะมีการแสดงข้อความทางจอภาพดังนี้

Number 1

Number 2

Number 3

Number 4

Number 5

End Program

จากตัวอย่างคำสั่งในการทำซ้ำในลูปมีเพียงคำสั่งเดียวเราสามารถเขียนโปรแกรมให้สั้นขึ้นได้โดยไม่ต้องใส่เครื่องหมาย { และ } คอมไพเลอร์จะทำซ้ำเฉพาะคำสั่งที่อยู่ถัดจาก for ในที่นี้จะไม่ทำซ้ำคำสั่ง printf("End Program")

```
int i;           //ประกาศตัวแปร i เป็นจำนวนเต็ม
for(i = 1; i <= 5; i++)
    printf("Number %d\n",i);
printf("End Program");
```

ผลลัพธ์ที่ได้จะเหมือนกับตัวอย่างแรกแต่ผู้เขียนไม่แนะนำให้ใช้วิธีหลังเพราะจะทำให้ผิดพลาดได้ง่าย

ตัวอย่างที่ผ่านมาจะเป็นการทำซ้ำโดยที่เพิ่มค่าตัวแปรในการทำซ้ำแล้วตรวจสอบเงื่อนไขการทำซ้ำ เราสามารถทำซ้ำโดยการลดค่าตัวแปรการทำซ้ำได้ดังนี้

```
int i;           //ประกาศตัวแปร i เป็นจำนวนเต็ม
for(i = 5; i >= 1; i--)
{
    printf("Number %d\n\r",i);
}

printf("End Program");
```

เป็นการวนรอบโดยเริ่มต้นให้ตัวแปร i มีค่าเท่ากับ 5 โดยแต่ละรอบของการทำซ้ำจะมีการลดค่า i อีก 1 ด้วยคำสั่ง i-- และจะทำซ้ำต่อเมื่อ i ยังมากกว่าหรือเท่ากับ 1 ซึ่งก็คือจะหยุดทำซ้ำเมื่อ i ลดค่าจนเท่ากับ 1 นั่นเอง

ผลลัพธ์

จะมีการแสดงข้อความทางจอภาพดังนี้

Number 5

Number 4

Number 3

Number 2

Number 1

End Program

นอกจากตัวอย่างที่กล่าวไปแล้วนั้นที่เพิ่มหรือลดค่าตัวแปรที่ใช้ตรวจสอบการทำซ้ำครั้ง 1 เราสามารถเพิ่มหรือลดค่าคราวละมากกว่า 1 ได้โดยใช้

```
for(i = 0; i <= 10; i = i+2)
{
    printf("Number %d\n\r",i);
}

printf("End Program");
```

และ

```
for(i = 10; i >= 0; i = i-2)
{
    printf("Number %d\n",i);
}
printf("End Program");
```

ซึ่งจะเป็นการเพิ่มและลดค่าตัวแปรคราวละ 2 ตามลำดับ

Note ฟังก์ชัน printf()

ใน คอมไพเลอร์ CCS C ฟังก์ชัน printf() เป็นฟังก์ชันที่ใช้ในการแสดงผลออกทางหน้าจอคอมพิวเตอร์เมื่อไมโครคอนโทรลเลอร์เชื่อมต่อกับคอมพิวเตอร์ผ่านพอร์ตอนุกรม โดยก่อนใช้ฟังก์ชัน printf() จะต้องมีไคลร์กทีฟ # use rs232(option) เช่น

```
#use rs232(baud = 9600, xmit = PIN_C6, rcv = PIN_C7)
```

เป็นการประกาศใช้งานการสื่อสารแบบอนุกรม แบบ rs – 232 โดยมีค่า Baud Rate (อัตราการถ่ายทอดข้อมูล มีหน่วยเป็นบิตต่อวินาที (bit per second: bps)) และกำหนดให้ขาที่ใช้ในการส่งสัญญาณเป็น PIN_C6 และขารับสัญญาณเป็น PIN_C7 ซึ่งในการกำหนดขารับส่งสัญญาณต้องกำหนดให้ตรงตามคู่มือของไมโครคอนโทรลเลอร์แต่ละบอร์ดด้วย

รูปแบบการทำงานของฟังก์ชัน printf()

```
printf(control code, argument variable);
```

เมื่อ argument variable คือ ค่าคงที่, ค่าตัวแปร หรือ คำนิพจน์ที่ต้องการแสดง ถ้ามีมากกว่า 1 ค่าต้องใช้เครื่องหมายคอมม่า (,) คั่นแยก ซึ่งหากเป็นการแสดงเพียงข้อความจะไม่มีในส่วนนี้ เช่น

```
printf("End Program");
```

control เป็นข้อความที่ต้องการแสดงและรหัสรูปแบบการแสดงผลของค่า argument variable โดยที่ control จะต้องอยู่ภายในเครื่องหมาย “ “ โดยต้องมีรหัสรูปแบบและรหัสควบคุมการแสดงผลไว้ภายในด้วยหากเป็นการแสดงค่าของตัวแปร เช่น

```
printf("Number %d\n",i);
```

เป็นการแสดงข้อความ Number เว้นวรรคหนึ่งครั้งตามด้วยค่าของตัวแปร i ด้วยรูปแบบการแสดงผลแบบจำนวนเต็มแบบคิดเครื่องหมาย จากนั้นให้เลื่อนเคอร์เซอร์ไปยังตำแหน่งแรกของบรรทัดถัดไป

รหัสรูปแบบและควบคุมการแสดงผล	การใช้งาน
%c	ใช้กับการแสดงผลตัวแปรที่เก็บค่าอักขระตัวเดียว (Character)
%s	ใช้กับการแสดงผลตัวแปรที่เก็บค่าข้อความ
%d	ใช้กับการแสดงผลตัวแปรที่เก็บค่าจำนวนเต็ม
%u	ใช้กับการแสดงผลตัวแปรที่เก็บค่าจำนวนเต็มบวก
%f	ใช้กับการแสดงผลตัวแปรที่เก็บค่าเลขทศนิยม
%e	ใช้กับการแสดงผลตัวแปรที่เก็บค่าเลขทศนิยมในรูปแบบ exponential
%x	ใช้กับการแสดงผลตัวแปรจำนวนเต็ม 8 บิตในรูปแบบเลขฐานสิบหกตัวพิมพ์เล็ก
%X	ใช้กับการแสดงผลตัวแปรจำนวนเต็ม 8 บิตในรูปแบบเลขฐานสิบหกตัวพิมพ์ใหญ่
%lx	ใช้กับการแสดงผลตัวแปรจำนวนเต็ม 16 บิตในรูปแบบเลขฐานสิบหกตัวพิมพ์เล็ก
%lX	ใช้กับการแสดงผลตัวแปรจำนวนเต็ม 16 บิตในรูปแบบเลขฐานสิบหกตัวพิมพ์ใหญ่
%lu	ใช้กับการแสดงผลตัวแปรจำนวนเต็มบวกขนาด 16 บิต
%ld	ใช้กับการแสดงผลตัวแปรจำนวนเต็มขนาด 16 บิต
\n	ขึ้นบรรทัดใหม่
\t	เว้นระยะ 6 ตัวอักษรในแนวนอน
\r	ให้เลื่อนเคอร์เซอร์ไปยังตำแหน่งแรกของบรรทัด

\b	ให้เลื่อนเคอร์เซอร์ย้อนหลังกลับไป 1 ตัวอักษรพร้อมกับลบตัวอักษร
----	--

คำสั่งการวนรอบ While

การทำงานของการทำงานซ้ำแบบ while จะทำการตรวจสอบเงื่อนไขก่อนว่าเป็นจริงหรือไม่ ถ้าเป็นจริงก็จะทำคำสั่งที่อยู่ภายในลูปจนกระทั่งเงื่อนไขที่กำหนดการทำงานซ้ำจะเป็นเท็จ

รูปแบบการใช้คำสั่ง

```
while(เงื่อนไข)
{
    ชุดคำสั่ง;
}
```

ตัวอย่าง

```
n = 1;
while(n<=5)
{
    printf("Science Robot Episode 1 Line Tracking %d\n\r",n);
    n = n + 1;
}
```

ผลลัพธ์

Science Robot Episode 1 Line Tracking 1

Science Robot Episode 1 Line Tracking 2

Science Robot Episode 1 Line Tracking 3

Science Robot Episode 1 Line Tracking 4

Science Robot Episode 1 Line Tracking 5

ในหลายๆ ครั้งในการเขียนโปรแกรมจะต้องมีการวนรอบเพื่อทำงานหลักอยู่ตลอดเวลาเราสามารถทำได้โดย

```
while(true)
{
    ชุคคำสั่ง;
}
```

คำสั่งการวนรอบ do while

คำสั่งการวนรอบแบบ do...while จะแตกต่างกับการวนรอบแบบ while ตรงที่คำสั่ง do...while จะทำการทำคำสั่งภายในลูปก่อนที่จะตรวจสอบเงื่อนไข ดังนั้นจะมีการทำงานคำสั่งภายในลูปอย่างน้อย 1 ครั้ง หากตรวจสอบเงื่อนไขแล้วเป็นจริงก็จะทำการวนทำซ้ำต่อแต่หากเงื่อนไขเป็นเท็จก็จะหยุดการทำซ้ำ

รูปแบบการใช้คำสั่ง

```
do
{
    ชุคคำสั่ง;
} while(เงื่อนไข)
```

ตัวอย่าง

```
n = 1;
do
{
    printf("Science Robot Episode 1 Line Tracking %d\n\r",n);
    n = n + 1;
} while(n<=5)
```

ผลลัพธ์

Science Robot Episode 1 Line Tracking 1

Science Robot Episode 1 Line Tracking 2

Science Robot Episode 1 Line Tracking 3

Science Robot Episode 1 Line Tracking 4

Science Robot Episode 1 Line Tracking 5

```
ถ้า      n = 5;
        do
        {
            printf("Science Robot Episode 1 Line Tracking %d\n\r",n);
            n = n + 1;
        } while (n<=5)
จะได้ผลลัพธ์
```

Science Robot Episode 1 Line Tracking 1

แต่หากเป็นคำสั่งทำซ้ำแบบ while จะไม่มีการทำคำสั่งภายในลูป

การสร้างฟังก์ชัน

ในการเขียนโปรแกรมหลายครั้งที่ต้องใช้คำสั่งหรือทำงานอย่างใดอย่างหนึ่งบ่อยๆ ถ้าเราต้องเขียนชุดคำสั่งเหล่านั้นใหม่ทุกครั้งก็จะทำให้โปรแกรมของเรามีความยาวเกินความจำเป็นและไม่สะดวกในการปรับปรุงและแก้ไขโปรแกรม เราสามารถแก้ปัญหานี้ได้โดยการนำชุดคำสั่งเหล่านั้นมารวมเป็นฟังก์ชัน โดยการสร้างฟังก์ชันขึ้นใช้งานนั้นมีวิธีดังนี้

Type Function Name (Parameter List)

```
{
    ชุดคำสั่ง;
    [return];
}
```

เมื่อ

Type คือ ประเภทของข้อมูลที่มีการส่งค่ากลับจากฟังก์ชัน ในกรณีที่ฟังก์ชันไม่มีการส่งค่ากลับส่วนนี้จะใช้คำสงวน void

Function Name คือ ชื่อของฟังก์ชัน โดยหลักการตั้งชื่อจะเป็นแบบเดียวกับการตั้งชื่อตัวแปรดังที่ได้กล่าวไปแล้ว

Parameter List คือ ตัวแปรที่ใช้รับค่าเข้ามาในฟังก์ชัน ในส่วนนี้หากไม่มีการส่งค่าผ่านมายังฟังก์ชันก็จะว่างไว้ หรือจะใส่ void แทนก็ได้

return ใช้ในการส่งค่ากลับโดยหลังจากคำสั่งนี้จะตามด้วยค่าหรือตัวแปรที่ต้องการส่งค่ากลับ ในส่วนนี้จะไม่มีถ้าฟังก์ชันไม่มีการคืนค่า

ตัวอย่าง

```
void Show()
{
    int i ;
    for(i = 1; i <= 20; i++)
    {
        printf("Science Robot Episode 1 Line Tracking\n\r");
    }
} //จบฟังก์ชัน
```

เป็นฟังก์ชันที่ใช้แสดงข้อความ Science Robot Episode 1 Line Tracking จำนวน 20 ครั้ง และเป็นฟังก์ชันที่ไม่มีการคืนค่ากลับ ส่วนวิธีการใช้งานฟังก์ชันสามารถทำได้ดังนี้ เช่น

```
main()
{
    ...
    Show();    // เรียกใช้ฟังก์ชัน Show()
    ...
} //จบฟังก์ชันหลัก
```


ต่อไปเป็นตัวอย่างของการสร้างฟังก์ชันที่มีการคืนค่า

```
int ADD_AB(int A, int B)
{
    int result;
    result = A + B;
    return result; // คืนค่าผลลัพธ์
} //จบฟังก์ชัน
```

เป็นฟังก์ชันที่ใช้บวกเลขจำนวนเต็มสองค่าแล้วทำการคืนค่าผ่านตัวแปร result ไปยังตัวแปรที่รับค่าจากฟังก์ชัน การเรียกใช้ฟังก์ชันประเภทนี้สามารถทำได้โดย เช่น

```
main()
{
    int add; //ประกาศตัวแปร add เป็นตัวแปรชนิดจำนวนเต็ม
    add = ADD_AB(2,3);
    printf("A+B = %d\n",add);
}
```

จาก add = ADD_AB(2,3) คำสั่งนี้จะเริ่มทำจากทางด้านซ้ายมือของเครื่องหมายเท่ากับก่อน โดยเรียกใช้ฟังก์ชันการบวกเลขจำนวนเต็มสองจำนวนแล้วนำค่าที่ได้มาเก็บในตัวแปร add สังเกตค่าภายในวงเล็บของคำสั่ง ADD_AB(2,3) กับ int ADD_AB(int A, int B) ค่าแรกซึ่งเท่ากับ 2 จะถูกส่งไปยังตัวแปร A และค่าที่สองซึ่งมีค่าเท่ากับ 3 จะถูกส่งไปยังตัวแปร หลังจากนี้จะเข้าสู่การทำงานของฟังก์ชัน ADD_AB(int A, int B) ซึ่งภายในฟังก์ชันจะทำการบวกค่า A กับค่า B แล้วนำไปเก็บในตัวแปร result หลังจากนั้นก็คืนทำการคืนค่ากลับมายังตัวแปร add จากนั้นจะมาทำคำสั่ง printf("A+B = %d\n",add) ก็จะแสดงผลลัพธ์ทางจอภาพ ดังนี้

A+B = 5

สังเกต การเรียกใช้งานฟังก์ชันที่มีการคืนค่า จะต้องมีตัวแปรที่ใช้ในการรับค่าจากฟังก์ชันที่เรียกใช้เสมอ ในที่นี้ คือ ตัวแปร add

อาร์เรย์ และ สตริง

ในการเขียนโปรแกรมด้วยภาษาซีบางครั้งเราอาจจำเป็นต้องใช้งานตัวแปรชนิดเดียวกันหลายๆ ตัว เช่น เราต้องการใช้ตัวแปรชนิดจำนวนเต็ม จำนวน 4 ตัว ซึ่งเราสามารถทำได้ดังนี้

```
int x1, x2, x3, x4;
```

หากต้องการหลายๆ ตัวมากกว่านี้อาจยุ่งยากและไม่สะดวกในการจัดการ เพื่อความสะดวกภาษาซีได้จัดเตรียมวิธีการจัดเก็บข้อมูลชนิดเดียวกันหลายๆ ข้อมูลโดยใช้ตัวแปรแบบ อาร์เรย์ (array) การประกาศตัวแปรแบบอาร์เรย์สามารถทำได้ดังนี้

ประเภทข้อมูล ชื่ออาร์เรย์[ขนาดอาร์เรย์]

หากเป็นการประกาศอาร์เรย์แบบสองมิติสามารถประกาศได้ดังนี้

ประเภทข้อมูล ชื่ออาร์เรย์[ขนาดอาร์เรย์] [ขนาดอาร์เรย์]

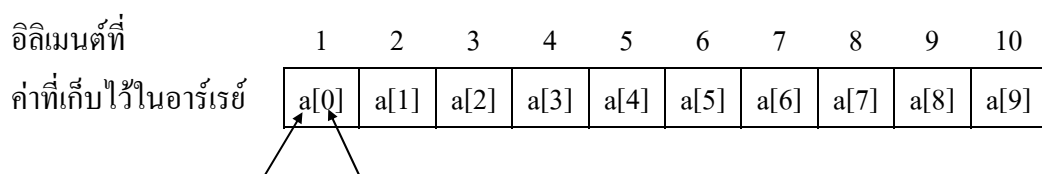
หากต้องการประกาศอาร์เรย์ที่มีหลายมิติสามารถประกาศได้ดังนี้

ประเภทข้อมูล ชื่ออาร์เรย์[ขนาดอาร์เรย์] [ขนาดอาร์เรย์].....[ขนาดอาร์เรย์]

ตัวอย่างการประกาศตัวแปรชนิดอาร์เรย์ เช่น

```
int a[10];
```

จากตัวอย่างเป็นการเป็นการประกาศตัวแปรชื่อ a จำนวน 10 อิลิเมนต์ (Element) แต่ละอิลิเมนต์จะจัดเก็บข้อมูลชนิดจำนวนเต็ม การอ้างถึงอาร์เรย์ a แต่ละอิลิเมนต์จะใช้ดัชนี (index) เป็นตัวอ้างถึงซึ่งแสดงเป็นแผนภาพได้ดังนี้



ชื่อตัวแปรอาร์เรย์ ดัชนีบอกตำแหน่ง(อยู่ภายในเครื่องหมาย [])

เราสามารถกำหนดค่าของแต่ละอิลิเมนต์ได้ เช่น

a[0] = 25;

ค่าที่เก็บอยู่ในอิลิเมนต์ที่ 1 ก็จะมีค่าเท่ากับ 25 นอกจากนี้เราสามารถกำหนดค่าอาร์เรย์ตอนประกาศตัวแปรได้ด้วย เช่น

int a [10] = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};

ก็จะได้ตัวแปรชนิดอาร์เรย์ ชื่อ a เก็บข้อมูลจำนวนเต็มจำนวน 10 ค่า ดังแผนภาพ

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]
2	4	6	8	10	12	14	16	18	20

เราสามารถนำค่าที่อยู่ในแต่ละอิลิเมนต์มาคำนวณหรือนำมาใช้ได้ เช่น

x = a[4]*3;

เป็นการนำค่าที่เก็บในตัวแปรอาร์เรย์ออกมาใช้งาน ในที่นี้ a[4] มีค่าเท่ากับ 10 เมื่อนำมาคูณด้วย 3 แล้วนำไปเก็บในตัวแปร x ดังนั้น x จะมีค่าเท่ากับ 30 นอกจากนี้เรายังสามารถแก้ไขข้อมูลที่อยู่ภายในตัวแปรชนิดอาร์เรย์ได้ถ้าตัวแปรอาร์เรย์ไม่ถูกประกาศเป็นค่าคงที่ เช่น

a[9] = a[1]*10;

จะทำให้ค่าที่จัดเก็บในตัวแปร a ดัชนีที่ 9 มีการเปลี่ยนแปลงซึ่งจะได้

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]
2	4	6	8	10	12	14	16	18	40

ตัวแปรแบบสตริง

ในภาษาซีจะไม่มีชนิดของข้อมูลแบบสตริง แต่จะมีเพียงตัวแปรอักขระก็สามารถเก็บตัวอักขระได้เพียง 1 ตัว เท่านั้น ดังนั้นในการที่จะจัดเก็บข้อมูลอักขระหลายๆ ตัวเพื่อทำเป็นข้อความนั้นต้องใช้ตัวแปรชนิดอาร์เรย์เข้ามาใช้งาน ดังนั้น สตริงในภาษาซี ก็คือ ตัวแปรแบบอาร์เรย์ที่จัดเก็บอักขระขนาด 1 มิติ นั่นเอง ดังนั้นการใช้งานสตริงก็ต้องประกาศตัวแปรอาร์เรย์ชนิดอักขระ (char) เช่น

```
char name[9];
```

เป็นการประกาศอาร์เรย์เก็บข้อมูลชนิดอักขระจำนวน 9 อลิเมนต์ ข้อแตกต่างของการใช้ตัวสตริงกับอาร์เรย์ชนิดอื่นคือ ตัวแปรสตริงจะต้องปิดท้ายด้วย null character (ซึ่งมีรหัส แอสกีเท่ากับ \0) เพื่อเป็นการบอกให้คอมพิวเตอร์ทราบว่าสิ้นสุดสตริงแล้ว ดังนั้นจากตัวอย่างจะเห็นว่าสามารถเก็บข้อมูลสตริงได้จำนวน 8 ตัวอักษรหรือ 8 อักขระนั่นเอง เพราะอิลิเมนต์สุดท้ายจะเป็น null character เราสามารถกำหนดค่าเริ่มต้นให้กับสตริงได้ดังนี้ เช่น

```
char name[9] = "WERAPONG";
```

สามารถแสดงการจัดเก็บได้ดังแผนภาพ

อิลิเมนต์ที่	1	2	3	4	5	6	7	8	9
ค่าที่จัดเก็บ	W	E	R	A	P	O	N	G	\0

เราสามารถกำหนดค่าทีละ 1 อักขระได้เช่น

```
name[1] = 'A';
```

จะได้

อิลิเมนต์ที่	1	2	3	4	5	6	7	8	9
ค่าที่จัดเก็บ	W	A	R	A	P	O	N	G	\0

คอมไพเลอร์ CCS C ได้จัดเตรียมฟังก์ชันการจัดการเกี่ยวกับสตริงไว้หลายฟังก์ชันไม่ว่าจะเป็นการเปรียบเทียบสตริง การคัดลอก หรือการแสดงผล ซึ่งจะไม่กล่าวในที่นี้สามารถศึกษาเพิ่มเติมได้จาก help ของคอมไพเลอร์ CCS C

ตัวแปรแบบพอยน์เตอร์

ในที่นี้จะกล่าวถึงเฉพาะความสัมพันธ์ระหว่างตัวแปรอาร์เรย์กับตัวแปรชนิดพอยน์เตอร์เท่านั้น ที่ผ่านมาในการอ้างถึงข้อมูลในตัวแปรแบบอาร์เรย์เราจะใช้ตัวดัชนีในการอ้างถึง นอกจากนี้ เรายังสามารถใช้ตัวแปรแบบพอยน์เตอร์ในการอ้างถึงข้อมูลนี้ได้ด้วย ตัวดำเนินการที่เราจำเป็นต้องทำความรู้จักเกี่ยวกับตัวแปรแบบพอยน์เตอร์ คือ & และ *

ตัวอย่างเช่น มีการประกาศตัวแปร

```
char j;
```

จะเป็นการประกาศตัวแปรสำหรับเก็บตัวแปรอักขระซึ่งก็就会有มีการจองหน่วยความจำขนาด 8 บิต โดยจะมีแอดเดรสระบุตำแหน่งการจัดเก็บข้อมูลเพื่อสามารถอ้างถึงตัวแปร j ได้ เราสามารถอ้างถึงแอดเดรสของตัวแปร j ได้โดยใช้ตัวดำเนินการ &j การประกาศตัวแปรพอยน์เตอร์สำหรับเก็บแอดเดรสตัวแปร j สามารถทำได้ดังนี้

```
char j;
```

```
char *dptr;
```

```
dptr = &j;
```

เป็นการประกาศตัวแปร dptr เป็นแบบตัวแปรพอยน์เตอร์และให้ชี้ไปยังแอดเดรสตัวแปร j สำหรับตัวแปรแบบอาร์เรย์เช่น

```
int ab[5] = {2, 4, 6, 8, 10};
```

ปกติเราจะอ้างถึงข้อมูลแต่ละอิลิเมนต์ได้ด้วยการใช้ดัชนี เช่น a[0] เป็นการอ้างถึงอิลิเมนต์ที่ 1 ของตัวแปร a เราสามารถใช้ตัวแปรแบบพอยน์เตอร์แทนได้โดยให้ตัวแปรแบบพอยน์เตอร์ชี้ไปยังดัชนีหรืออิลิเมนต์แรกแล้วทำการเพิ่มค่าพอยน์เตอร์ได้ เช่น

```

int x;

int *ip;          // ประกาศตัวแปร ip เป็นแบบพอยน์เตอร์
ip = &ab[0];      // ตัวแปร ip จะชี้ไปที่ตำแหน่ง ab[0]
x = *ip;          // นำค่า ab[0] มาเก็บไว้ในตัวแปร x

```

จะได้ x มีค่าเท่ากับ 10 การเลื่อนค่า ดัชนีเราสามารถทำได้โดยการเพิ่มค่า ip เช่นหากตอนนี้ ip ชี้อยู่ที่ ab[0] เมื่อเพิ่มค่า

```
ip = ip + 1;
```

จะทำให้ชี้ไปที่ ab[1] ดังนั้นหากต้องการอ้างถึงอีลิเมนต์ที่ i หรือ a[i] สามารถทำได้โดยเพิ่มค่า ip = ip + i

พอร์ตอินพุต เอาท์พุต

การใช้ไมโครคอนโทรลเลอร์ติดต่อกับอุปกรณ์ภายนอกนั้นจะใช้พอร์ตในการเชื่อมต่อกับอุปกรณ์ภายนอก ถ้าต้องการส่งข้อมูลก็จะใช้เอาท์พุตพอร์ต ถ้าต้องการรับข้อมูลเข้ามาก็จะใช้อินพุตพอร์ต ตัวอย่างการใช้งานเอาท์พุตพอร์ต เช่น ใช้ไมโครคอนโทรลเลอร์สร้างไฟวิง โดยต่อวงจรดังรูปที่ 20 ซึ่ง LED จะสว่างเมื่อเราสั่งให้พอร์ตเอาท์พุต (ในที่นี้ คือ พอร์ต C) เป็นลอจิกสูง



รูปที่ 20 แสดงการต่อ LED กับพอร์ตเอาท์พุต

ตัวอย่างโปรแกรมไฟวิง

```

#include<16F877.h>

#define delay (clock = 4000000)

#define fuses XT, NOWDT

```

```
int LED[8] = {0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80};
```

```
void main(void)
{
    int i;
    while (true)
    {
        for(i=0; i<=7; i++)
        {
            output_c(LED[i]);
            delay_ms(500);
        }
    }
} //จบฟังก์ชัน
```

จากตัวอย่างจะเป็นการสร้างไฟวิ่งโดยให้ไฟติดทีละดวงไปเรื่อยๆ จนครบ 8 ดวง ฟังก์ชันที่เกี่ยวข้องกับพอร์ตเอาต์พุต คือ

output_x (value)

โดยที่ x จะแทนพอร์ต เช่น output_c(value) ก็จะเป็นการใช้งานพอร์ตเอาต์พุตโดยใช้พอร์ต C ส่วน value จะเป็นค่าที่ต้องการส่งออกไปยังพอร์ต

output_high(PIN)

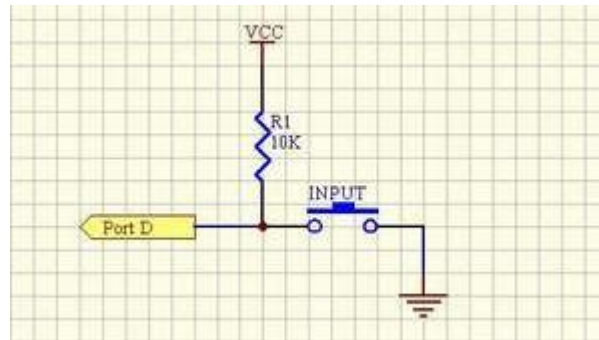
output_low(PIN)

จะเป็นคำสั่งเอาต์พุตลอจิกสูง (5 โวลต์) และ ลอจิกต่ำ (0 โวลต์) ตามลำดับ โดยที่ PIN คือ ขาที่ต้องการส่งเอาต์พุตออกไป เช่น output_high(PIN_C0) จะเป็นคำสั่งให้ส่งลอจิกสูงไปยังพอร์ต C ขา C0 หรือ output_low(PIN_C2) จะเป็นคำสั่งในการส่งลอจิกต่ำไปยังพอร์ต C ขา C2

ส่วนการใช้งานใช้งานพอร์ตอินพุตจะมีการใช้งานอยู่สองรูปแบบ คือ พอร์ตอินพุตแบบ ดิจิตอลและพอร์ตอินพุตแบบอนาล็อก พอร์ตอินพุตแบบดิจิตอล อินพุตมีได้แค่สองค่าเท่านั้นคือ 0

โวลต์ และ 5 โวลต์เท่านั้น ส่วนการใช้งานพอร์ตอินพุตแบบอนาล็อกอินพุตจะเป็นสัญญาณที่มีค่าระหว่างจาก 0 ถึง 5 โวลต์

ตัวอย่างการใช้งานพอร์ตอินพุตแบบดิจิทัลที่เห็นได้ชัด เช่น การรับค่าสวิตช์ถ้ามีการกดสวิตช์แล้วให้ทำงานอย่างใดอย่างหนึ่ง ตัวอย่างที่มักใช้งานในหุ่นยนต์ เช่น ใช้สวิตช์ในการตรวจสอบการชน เมื่อมีการชนสวิตช์ก็จะถูกกดซึ่งเมื่อได้รับสัญญาณก็จะสั่งให้หุ่นยนต์หลบสิ่งกีดขวาง เป็นต้น



รูปที่ 21 แสดงการต่อสวิตช์กับอินพุตพอร์ต

ฟังก์ชันที่เกี่ยวข้องกับอินพุตพอร์ตดิจิทัลมีดังนี้

`input_x()`

โดยที่ x โดยที่ x จะแทนชื่อพอร์ตใดๆ เช่น `input_c()` จะเป็นการอ่านค่าลอจิกที่ต่ออยู่กับพอร์ต C ทั้งหมดทุกขา ลักษณะการใช้งานนั้นจะต้องมีตัวแปรในการรับค่าด้วย เช่น

`data = input_c();`

เป็นการอ่านค่าลอจิกที่ต่ออยู่กับพอร์ต C ทั้งหมดทุกขามาเก็บไว้ในตัวแปร data

`input(PIN)`

เป็นคำสั่งในการอ่านค่าอินพุตตามขาที่กำหนด เช่น `input(PIN_C0)` จะเป็นการรับค่าอินพุตจาก PIN_C0 การใช้งานจะต้องมีตัวแปรในการรับค่า เช่น


```
status = input(PIN_C0);
```